Mini Project 3

The Governed AI Pipeline

Mohammad Alrashed

Claude Code Mastery — AI-Augmented Software Engineering

Week 3 · Sessions 5 + 6

12 Apr – 18 Apr 2026

GitHub Repository · Live Dashboard

Table of Contents

- 1. Executive Summary
- 2. Part 1 — Workflow Mapping & Automation Leverage
- 3. Part 2 — Slash Command Pipeline
- 4. Part 3 — Governance & Safety
- 5. Part 4 — Measurement & ROI
- 6. Thinking Questions (Q1–Q7)
- 7. Tactical Questions (Q8–Q12)
- 8. Deliverables Checklist
- 9. Governance Playbook Appendix

1. Executive Summary

The Governed AI Pipeline reduces per-feature development cycle time from 227 minutes to 132 minutes — a **42% reduction** — by automating the five highest-leverage steps in the SDLC: test generation, code review, commit messaging, PR creation, and codebase onboarding.

For a 10-person team completing 5 features per developer per week at $150/hour, this translates to **$4,875 in weekly savings** and **$253,500 annually**. Beyond raw time savings, the pipeline delivers measurable quality improvements: 87% test coverage enforced by /test-gen (up from 60-70% baseline), zero secrets committed (enforced by check-secrets.py), consistent commit messages and PR descriptions, and a complete audit trail for compliance.

Three governance hooks blocked 100% of simulated dangerous operations in testing: rm -rf (validate-bash.py), a fake Stripe key (check-secrets.py), and an edit to a frozen file (scope-guard.sh). The pipeline is designed as a portable .claude/ directory that can be dropped into any repository.

2. Part 1 — Workflow Mapping & Automation Leverage

2.1 Current Workflow

The development workflow was mapped as a 12-step lifecycle from ticket triage to deployment. Each step was annotated with time (minutes), tools used, and pain points based on real Week 2 experience building the URL shortener.

#	Step	Time	Tools	Pain Level
1	Ticket Triage	10 min	GitHub Issues	Medium
2	Understand Codebase	25 min	IDE, grep, git blame	HIGH
3	Plan Implementation	15 min	Notes, mental model	Medium
4	Write Code	45 min	VS Code, Claude Code	Core work
5	Write Tests	30 min	Vitest	HIGH
6	Self-Review & Lint	15 min	ESLint, git diff	HIGH
7	Commit & Push	5 min	git CLI	Medium
8	Open PR	10 min	GitHub	Medium
9	Peer Review	45 min	GitHub PR UI	HIGH
10	Address Feedback	20 min	IDE, git	HIGH
11	Merge	2 min	GitHub	Low
12	Deploy	5 min	Render / GH Pages	Low

Total cycle time: 227 min (~3.8 hours) per feature. 5 of 12 steps are high-pain.

2.2 Automation Leverage Framework

Each step was scored on Frequency (1-10), Time per occurrence (1-10), and AI Capability (1-10). ROI = (Freq × Time × AI Cap) / 100, normalized to 1-10.

Step	Freq	Time	AI Cap	ROI
Ticket Triage	10	3	2	1
Understand Codebase	6	6	8	6
Plan Implementation	8	4	5	3
Write Code	10	8	5	5
Write Tests	10	7	9	9

Step	Freq	Time	AI Cap	ROI
Self-Review & Lint	10	4	9	8
Commit & Push	10	2	9	7
Open PR	8	3	9	8
Peer Review	6	8	6	6
Address Feedback	6	5	4	2
Merge	10	1	3	1
Deploy	5	2	3	1

2.3 Top 3 Automation Targets

Target 1: Test Generation (ROI 9/10) → /test-gen

Every code change needs tests (freq 10), each round consumes 25-30 min (time 7), and LLMs excel at generating test cases from function signatures (AI cap 9). In Week 2, writing 23 tests took longer than the implementation. Projected savings: 25 min/feature.

Target 2: Code Review (ROI 8/10) → /review

Self-review is universal but limited by blind spots. The Week 2 self-critique loop improved security scores from 6/10 to 9/10, proving systematic review catches real issues. Creates a multiplier by reducing peer review cycles. Projected savings: 12 min/feature.

Target 3: PR Creation (ROI 8/10) → /ship

/ship chains the entire pipeline into one invocation. Every PR follows the same quality bar with structured summary, test coverage data, and governance metadata. Projected savings: 8 min/feature.

3. Part 2 — Slash Command Pipeline

3.1 CLAUDE.md Conventions (Key Excerpts)

- **Commit format:** week-3: <description> (lowercase, imperative, max 72 chars)
- **Testing:** Vitest, 80% coverage minimum, <module>.test.ts naming
- **Code style:** const by default, async/await, functions under 40 lines, no var
- **Security:** No hardcoded keys, no sk-ant- strings, no .env committed
- **File scope:** Edits only in week-3/, index.html. Frozen: week-1/, week-2/, docs/
- **Cost policy:** $5/session normal, $15 for /ship, $50/week/developer limit

3.2 /review — AI Code Review

- Reads staged diff, checks against CLAUDE.md conventions
- Outputs structured findings: file, line, severity, rationale, fix
- Verdict: PASS (no errors) or FAIL (errors found)
- /ship aborts if /review returns FAIL

3.3 /test-gen — Test Generation

- Identifies changed source files from git diff
- Generates vitest tests: happy path + error cases + edge cases
- Runs tests with --coverage, saves output to audit/
- Reports pass/fail summary with coverage percentage

3.4 /commit — Smart Commit Message

- Analyzes staged diff for type (feat/fix/docs/test/refactor/chore)
- Derives scope from changed paths
- Generates Conventional Commit: type(scope): imperative summary
- Includes body explaining what and why, references REQ-IDs

3.5 /ship — Full Pipeline

- Stage 1: /review — abort on error-severity findings
- Stage 2: /test-gen — abort on test failures
- Stage 3: /commit — create Conventional Commit
- Stage 4: gh pr create — structured PR with review + coverage data
- Records results to audit/ship-sample-run.md

3.6 /onboard — New Hire Onboarding

- Reads CLAUDE.md + README + source files
- Generates: architecture map, key files, conventions, test patterns
- Documents non-obvious invariants (route order, soft delete, store)
- Output as clean markdown for team wiki

3.7 /ship Pipeline Step Order Rationale

/review precedes /test-gen because: (1) fail-fast economics — review is faster than test generation, catching problems cheaply before expensive operations; (2) test quality depends on code quality — if code has missing validation, tests would test wrong behavior; (3) review findings inform what code paths need attention before coverage measurement.

4. Part 3 — Governance & Safety

4.1 Validation Hooks

validate-bash.py (PreToolUse, matcher: Bash)

Reads JSON from stdin, extracts command. Blocks: rm -rf, DROP TABLE, DROP DATABASE, git push --force, git reset --hard, chmod 777, curl|bash, wget|sh, AWS key patterns (AKIA...).

Blocked sample:

```
[2026-04-19T15:49:28Z] BLOCKED validate-bash.py "rm -rf /tmp/test" reason="rm -rf
(recursive force delete)"
```

check-secrets.py (PreToolUse, matcher: Edit|Write)

Scans content for: API keys, passwords, tokens, private keys, Stripe-style keys (sk_/pk_), GitHub PATs (ghp_), AWS access keys (AKIA). Case-insensitive regex matching.

Blocked sample:

```
[2026-04-19T15:49:31Z] BLOCKED check-secrets.py "test.ts" reason="Potential
Stripe-style key detected"
```

scope-guard.sh (PreToolUse, matcher: Edit|Write)

Extracts file_path from $CLAUDE_TOOL_INPUT. Allows: week-3/**, index.html. Blocks: .env, *.pem, *.key, credentials.json, week-1/**, week-2/**, docs/**, docker-compose.prod.yml, .github/workflows/**, root package.json.

Blocked sample:

```
[2026-04-19T15:49:33Z] BLOCKED scope-guard.sh "week-2/src/index.ts"
reason="frozen completed week"
```

4.2 Audit Logging

*audit-log.sh (PostToolUse, matcher: *)*

Sample audit.jsonl entry:

```
{"timestamp":"2026-04-19T15:49:40Z","user":"kewlf","branch":"main","tool":"Bash",
"input":"{\"command\":\"git status\"}","result":"On branch main"}
```

jq query — filter blocked Write actions:

```
jq 'select(.tool == "Write" and (.result | test("BLOCKED")))' audit.jsonl
```

*prompt-log.sh (UserPromptSubmit, matcher: *)*

Logs all user prompts to prompts.jsonl with timestamps.

*session-summary.sh (Stop, matcher: *)*

Generates session-summary.md with files modified, blocks triggered, tokens used, estimated cost. Appends cost entry to costs.jsonl.

4.3 Permission Configuration

Settings hierarchy: Enterprise > Project > User > Local. This file uses Project tier because governance rules should be shared across all developers, version-controlled, and peer-reviewed.

Rule	Type	Rationale
Read, Glob, Grep	Allow	Read-only, cannot modify state
Edit	Allow	Protected by check-secrets + scope-guard hooks
Bash(npm test), Bash(npx vitest*)	Allow	Test execution, side-effect-free
Bash(git status/diff/log)	Allow	Git read operations, cannot modify repo
Bash(gh pr*)	Allow	PR creation is the intended /ship output
Bash(rm *)	Deny	File deletion causes irreversible data loss
Bash(git push --force*)	Deny	Overwrites remote history, destroys work
Bash(git reset --hard*)	Deny	Discards local changes irreversibly
Bash(docker*)	Deny	Container ops out of scope for code editing
Bash(curl*|bash)	Deny	#1 supply chain attack vector
Bash(chmod 777*)	Deny	World-writable is a security anti-pattern

5. Part 4 — Measurement & ROI

5.1 Methodology

Controlled comparison using REQ-SHORT-006 (soft delete endpoint) under two conditions: baseline (manual workflow) vs. with-pipeline (/ship). Same task, same developer, same codebase.

5.2 Comparison

Metric	Baseline	Pipeline	Delta
Total time	89 min	35 min	-54 min (-61%)
Test coverage	78%	87%	+9 points
Commit quality	Manual	Conventional, auto	Consistent
PR quality	Adequate	Structured + governance	Standardized
Security scan	Manual	Automatic, every edit	Enforced
Audit trail	None	Full JSONL logs	Complete

5.3 Annual Projection (10-Person Team)

Line Item	Weekly	Annual
Direct time savings (10 devs × $675)	$6,750	$337,500
Pipeline cost (API + maintenance)	-$800	-$40,000
Net conservative savings	$5,950	$297,500
With bug-fix shift-left (midpoint)	$10,450	$253,500*

*Midpoint estimate uses conservative time savings plus 50% attribution on bug shift-left value.

5.4 Governance Controls Inventory

Control	Type	What It Prevents
validate-bash.py	PreToolUse	rm -rf, DROP TABLE, force push, curl|bash
check-secrets.py	PreToolUse	API keys, passwords, tokens, private keys
scope-guard.sh	PreToolUse	Edits to frozen weeks, .env, CI pipelines
audit-log.sh	PostToolUse	Records every action for compliance
prompt-log.sh	UserPromptSubmit	Records all prompts for review
session-summary.sh	Stop	Cost/activity summary per session

Control	Type	What It Prevents
permissions.allow	settings.json	Allowlist for safe operations
permissions.deny	settings.json	Denylist for dangerous patterns

6. Thinking Questions

Q1. What workflow steps did your Mermaid map reveal as highest-friction?

The map identified 5 of 12 steps as high-friction: Understand Codebase (25 min), Write Tests (30 min), Self-Review (15 min), Peer Review (45 min), and Address Feedback (20 min). These consume 135 of 227 total minutes (59%) but produce no new features. The highest-friction steps are also the most automatable: test generation (ROI 9), code review (ROI 8), and PR creation (ROI 8) all score high on AI capability because they follow predictable patterns LLMs handle well.

Q2. How does encoding conventions in CLAUDE.md change code review dynamics?

CLAUDE.md transforms review from subjective and inconsistent to deterministic and scalable. /review checks every diff against the same convention set every time, catching 'boring' violations systematically. This frees human reviewers to focus on architecture and business logic. In Week 2, the self-critique loop improved security scores from 6/10 to 9/10, proving systematic rule-checking catches real issues.

Q3. What did your 3 hook-triggered blocks demonstrate?

The blocks demonstrated that PreToolUse hooks provide prevention, not detection. validate-bash.py stopped rm -rf before execution; check-secrets.py blocked a key before it reached disk; scope-guard.sh protected a frozen file from modification. Post-hoc review would have found the damage after it occurred. The cost of prevention is near-zero (ms of hook execution) vs. hours of remediation.

Q4. Explain the design tradeoff between plan, default, and auto permission modes.

Plan mode: maximum safety, minimum velocity (every action needs approval). Auto mode: maximum velocity, relies entirely on hooks. We chose default mode: reads are free, writes/commands prompt the user. The hooks provide a second safety layer, but default mode adds defense-in-depth. The marginal velocity cost is small (a few extra confirmations) but the safety margin is significant.

Q5. What is the single most important ROI metric?

$253,500 annual net savings for a 10-person team. Pitch: developers spend 42% of feature time on process tasks AI can handle. Measured on a real task: manual 89 min vs pipeline 35 min (61% reduction). At 5 features/dev/week, 10 devs, $150/hr, net savings after API costs = $253,500/year. Plus: 87% test coverage enforced, zero leaked secrets, complete audit trail.

Q6. Describe two situations where a hook should be overridden.

1) Emergency hotfix to a frozen file: temporarily add to scope-guard allowlist, commit settings change with justification, revert after fix. 2) Legitimate rm -rf (e.g., corrupted node_modules): run directly in terminal (bypassing Claude Code) or temporarily disable the hook. Key principle: overrides should be auditable, not impossible. The hooks are guardrails, not prison walls.

Q7. How would your audit log support a post-incident investigation?

Example: a secret was committed despite check-secrets.py. Investigation via jq: (1) filter Write/Edit actions around suspected time, (2) search for the specific file, (3) cross-reference with prompt log for intent, (4) check blocked.log for the expected block. If no block entry exists, the regex patterns need updating. JSONL format enables fast jq queries with timestamp + tool + input providing the complete action timeline.

7. Tactical Questions

Q8. Show your /ship command's step order.

Step 1: /review (abort on errors) → Step 2: /test-gen (abort on failures) → Step 3: /commit (Conventional Commit) → Step 4: gh pr create (structured PR). /review precedes /test-gen for fail-fast economics, code quality dependency, and to ensure accurate coverage measurement.

Q9. Show one hook's settings.json entry and matching code.

settings.json entry:

```
{"matcher":"Bash","hook":"python3
week-3/.claude/hooks/validate-bash.py","description":"Block dangerous bash
commands"}
```

Interaction: Lifecycle (PreToolUse) = runs before execution. Matcher (Bash) = only triggers for Bash tool. Exit 0 = allow, exit 1 = block with stderr message shown to user. Fail-closed: any crash blocks the action.

Q10. Paste one audit.jsonl entry and a jq filter command.

```
{"timestamp":"2026-04-19T15:49:31Z","user":"kewlf","branch":"main","tool":"Write"
,"input":"{\"file_path\":\"test.ts\"}","result":"BLOCKED by check-secrets.py"}
```

```
jq 'select(.tool=="Write" and (.result|test("BLOCKED")) and
(.timestamp|startswith("2026-04-19")))' audit.jsonl
```

Q11. Show /test-gen coverage output and explain uncovered lines.

delete.ts: 78.57% stmts, 66.67% branch, 100% funcs, lines 18-24 uncovered. Lines 18-24 are the 404 error branch (short code not found). Tests exercise the happy path (successful delete → 204) but don't fully test the not-found case. Fix: add a test calling DELETE with a non-existent code and asserting 404.

Q12. Paste your full settings.json and explain each rule.

Allow rules: Read/Glob/Grep (read-only), Edit (protected by hooks), Bash(npm test/vitest) (side-effect-free), Bash(git status/diff/log) (read-only), Bash(gh pr*) (intended /ship output). Deny rules: Bash(rm *) (irreversible), git push --force (destroys history), git reset --hard (discards work), docker* (out of scope), curl*|bash (RCE vector), chmod 777 (security anti-pattern).

8. Deliverables Checklist

✓ .claude/commands/ — 5 slash commands (review, test-gen, commit, ship, onboard)

✓ .claude/hooks/ — 6 hooks (validate-bash, check-secrets, scope-guard, audit-log, prompt-log, session-summary)

✓ .claude/settings.json — permissions allow/deny, all 6 hooks wired

✓ .claude/audit/ — audit.jsonl, prompts.jsonl, costs.jsonl, blocked.log, session-summary.md, ship-sample-run.md

✓ CLAUDE.md — commit format, testing standards, file scope, security, cost policy

✓ docs/workflow-map.md — Mermaid flowchart with 12-step lifecycle annotations

✓ docs/leverage-analysis.md — ROI scoring table, top 3 targets, quadrant chart

✓ docs/roi-report.md — before/after comparison, annual projection, governance inventory

✓ docs/governance-playbook.md — 6-week team rollout plan with compliance mapping

✓ REPORT.md — answers to all 12 rubric questions with evidence

9. Governance Playbook Appendix

6-Week Team Rollout Plan

Week 1: Introduction: /onboard + /commit. Install Claude Code, opt-in commands, no hooks. Zero friction.

Week 2: Review + Tests: /review + /test-gen. Still opt-in. Collect time-per-PR metrics.

Week 3: Pipeline + Logging: /ship pipeline. Enable audit-log, prompt-log, session-summary. Start JSONL logging.

Week 4: Validation Hooks: Enable validate-bash, check-secrets, scope-guard. First blocking hooks. Test with deliberate triggers.

Week 5: Settings Hardening: Configure permissions allow/deny. Set --max-budget-usd and --max-turns limits. Cost tracking.

Week 6: Full Enforcement: /ship mandatory for all PRs. First compliance audit. ROI report from real data. Leadership presentation.

Compliance Mapping

Requirement	Control	Evidence
No secrets in source	check-secrets.py	blocked.log entries
No unauthorized edits	scope-guard.sh	blocked.log entries
No destructive commands	validate-bash.py	blocked.log entries
Complete audit trail	audit-log.sh	audit.jsonl (every action)
Prompt accountability	prompt-log.sh	prompts.jsonl
Cost tracking	session-summary.sh	costs.jsonl
Code review before merge	/review via /ship	PR body with findings
Test coverage minimum	/test-gen via /ship	Coverage report (80%+)
Consistent commits	/commit via /ship	Conventional Commits in git log
Permission least-privilege	settings.json	Documented allow/deny lists